

Adam Pinkos

CS 4040 – Design and Analysis of Algorithms

Dr. Majid Mirzanezhad

October 22nd, 2025

This write-up and program, created in Python, was to compare and analyze the two sorting algorithms Insertion Sort and Randomized Quicksort. The goal was to observe how the execution time of each algorithm changes as the input size increases and to verify whether the experimental results match their theoretical time complexities. To keep it simple, Insertion Sort is a quadratic-time algorithm with a time complexity of $O(n^2)$, making it efficient only for small or nearly sorted lists. Randomized Quicksort, on the other hand, uses a divide-and-conquer approach with an expected runtime of $O(n \log n)$ due to its random pivot selection, which helps avoid worst-case behavior on specific input patterns.

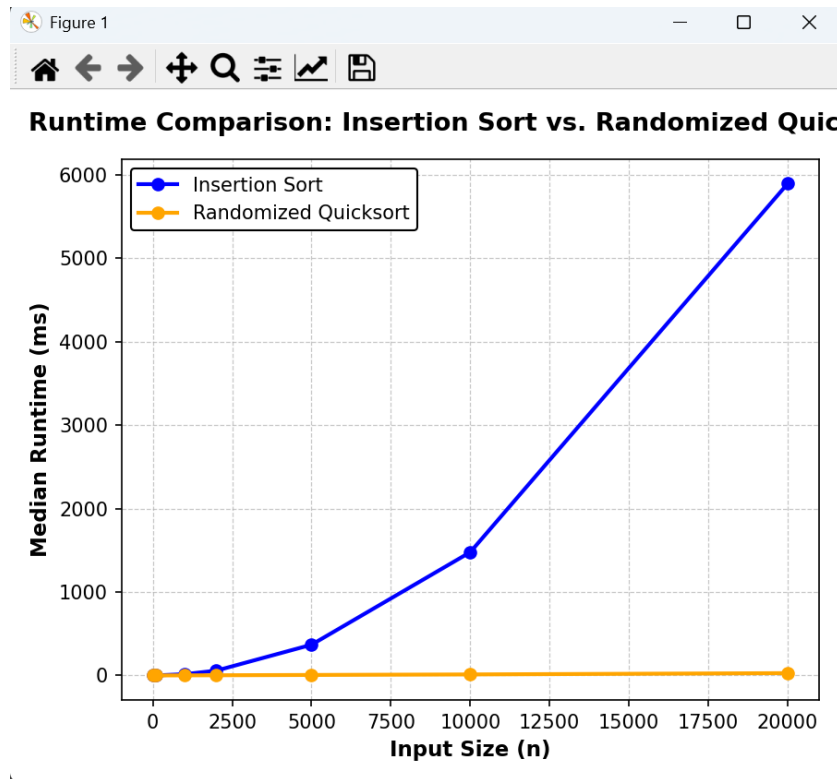
In this project, we used input sizes of ($n = 10, 100, 1000, 2000, 5000, 10000, 20000$), with five trials per size. A warm-up phase was included before.

```
C:\Users\al3pi\Downloads\CS4040>python main_runtime.py
n = 10 Insertion Sort median runtime (ms): 0.0
n = 10 Randomized Quicksort median runtime (ms): 0.01
n = 100 Insertion Sort median runtime (ms): 0.15
n = 100 Randomized Quicksort median runtime (ms): 0.08
n = 1000 Insertion Sort median runtime (ms): 16.8
n = 1000 Randomized Quicksort median runtime (ms): 1.15
n = 2000 Insertion Sort median runtime (ms): 59.61
n = 2000 Randomized Quicksort median runtime (ms): 2.09
n = 5000 Insertion Sort median runtime (ms): 375.53
n = 5000 Randomized Quicksort median runtime (ms): 5.79
n = 10000 Insertion Sort median runtime (ms): 1460.55
n = 10000 Randomized Quicksort median runtime (ms): 11.95
n = 20000 Insertion Sort median runtime (ms): 6024.65
n = 20000 Randomized Quicksort median runtime (ms): 26.34
Insertion Sort Tests Passed
Randomized Quicksort Tests Pass
```

This is the table (in the console) that I obtained when running the program. From these results, we can see that Insertion sort's runtime increases rapidly as the input size becomes large. In contrast, Randomized Quicksort maintains low runtimes across all tested input sizes. According to brilliant.org, insertion sort runtime is $O(n^2)$ for average and worse case. Also, according to

geeksforgeeks.org, the runtime for Randomize Quicksort is $O(n \log n)$. Not to mention, the slides from the class also state these. Therefore, based on the data, we see that it matches theoretical values.

Furthermore, I also made a graph that shows the runtimes of the two algorithms and what is happening at each step of the trials.



The graph clearly shows the performance difference between Insertion Sort and Randomized Quicksort as the input size increases. The blue line, representing Insertion Sort, rises sharply after around 2,000 elements and continues to climb steeply as the list size grows, reaching over 6,000 milliseconds by 20,000 elements. This steep, upward curve visually demonstrates the quadratic nature of Insertion Sort's runtime, meaning that as the input size doubles, the amount of work roughly quadruples.

The orange line represents Randomized Quicksort and shows that it is almost flat across all input sizes, indicating that its runtime increases very slowly. This flatness highlights the efficiency of the divide-and-conquer approach used by Randomized Quicksort, which breaks the input into smaller subproblems and sorts them independently using random pivot selection to avoid worst-case scenarios.

Sources Cited

Brilliant.org. (n.d.). *Insertion Sort*. Brilliant Math & Science Wiki. Retrieved from <https://brilliant.org/wiki/insertion/> (brilliant.org)

GeeksforGeeks. (2023, September 14). QuickSort using Random Pivoting

Retrieved from <https://www.geeksforgeeks.org/dsa/quicksort-using-random-pivoting/>

Mirzanezhad, M. (2025). All Lectures. CS 4040/5040 – Design and Analysis of Algorithms, Ohio University. [Class slides]